

Spring-23

4.

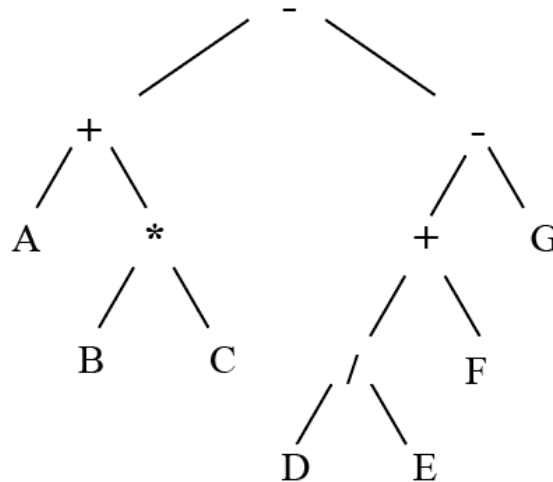
A. Construct an expression tree T for the arithmetic expression:

$$(A+B*C)-((D/E+F)-G)$$

Traverse the tree T in pre, post and inorder.

Solve:

Tree



1. Preorder Traversal (Root $\rightarrow$ Left $\rightarrow$ Right):

- + A * B C - + / D E F G

2. Postorder Traversal (Left $\rightarrow$ Right $\rightarrow$ Root):

A B C * + D E / F + G - -

3. Inorder Traversal (Left $\rightarrow$ Root $\rightarrow$ Right):

A + B * C - D / E + F - G

B. A binary tree T has 9 nodes. The inorder and preorder traversals of T yields the following sequences of nodes:

Inorder: E A C K F H D B G

Preorder: F A E K C D H G B

Draw the tree T.

Solve:

1. Root Selection:

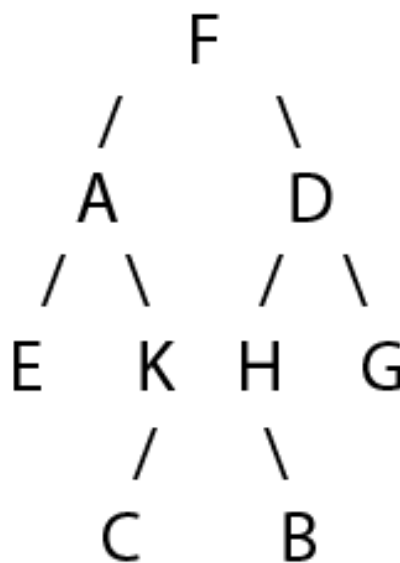
- The first node in the preorder sequence is the root. Hence, F is the root of the tree.

2. Divide Using Inorder:

- Use the inorder sequence to divide nodes into left and right subtrees:
 - Left subtree of F : E, A, C, K .
 - Right subtree of F : H, D, B, G .

3. Repeat the Process:

- For each subtree, use the next node in preorder as the root and split further using the corresponding inorder sequence. This gives:
 - A is the left child of F , with E as its left child and K as its right child (C is the left child of K).
 - D is the right child of F , with H as its left child and G as its right child (B is the left child of G).



C. What is a binary search tree? Suppose the following characters are inserted in order into an empty binary search tree.

B, A, N, G, L, A, D, E, S, H

i) Draw the tree T.

ii) Find the inorder traversal of T.

Solve:

A **binary search tree (BST)** is a binary tree where the left child contains smaller values, the right child contains larger values, and both subtrees are BSTs.

i) Draw the tree T



ii) Inorder traversal of T

A B D E G H L N S

OR

Illustrate the operation of Heap sort on the array $A=(5,13,2,25,N,7,17,20,8,4)$.

(Note: N is the sum of last two digits of your ID. i.e. if your ID is C151216, then $N=1+6=7$) **Page: 17-20**

Autumn 2023

4.

A. Analyze the differences between Binary tree and Complete binary tree with figure. Construct an expression tree T for the following arithmetic expression:

$$(A + B * C) + ((D / E - F) * G)$$

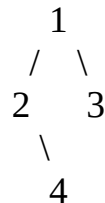
Traverse the tree T in preorder, postorder and inorder.

Solve:

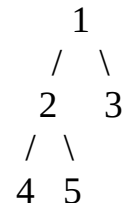
Differences Between Binary Tree and Complete Binary Tree:

Binary Tree	Complete Binary Tree
Each node has at most two children.	All levels are fully filled except possibly the last.
Nodes can be placed anywhere.	Nodes are filled from left to right.
May have gaps at any level.	Gaps are allowed only in the last level.

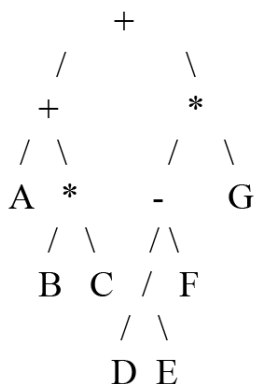
Binary Tree



Complete Binary Tree



$$(A + B * C) + ((D / E - F) * G)$$



Traverse the tree T:

1. Preorder

+, +, A, *, B, C, *, -, /, D, E, F, G

2. Postorder

A, B, C, *, +, D, E, /, F, -, G, *, +

3. Inorder

A, +, B, *, C, +, D, /, E, -, F, *, G

B. Suppose the following eight numbers are inserted in order into an empty binary search tree T: 50, 23, 44, 22, 77, 35, 60, 40

Draw the tree T and show each step.

Solve:

50, 23, 44, 22, 77, 35, 60, 40

Step-by-Step Insertion

1) ITEM = 50

50

2) ITEM = 23

50

/

23

3) ITEM = 44

50

/

23

\

44

4) ITEM = 22

50

/

23

/ \

22 44

5) ITEM = 77

50

/ \

23 77

/ \

22 44

6) ITEM = 50

50

/ \

23 77

/ \

22 44

/

35

7) ITEM = 50

50

/ \

23 77

/ \ /

22 44 60

/

35

8) ITEM = 50

50

/ \

23 77

/ \ /

22 44 60

/

35

\

40

OR

What do you mean by max heap? Build a max heap from the following list of numbers:

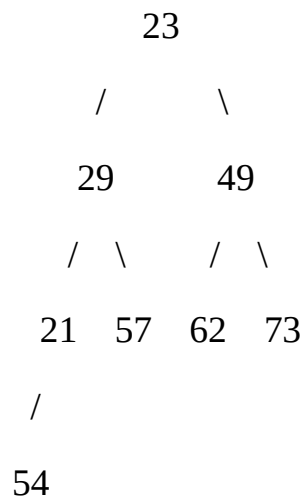
23, 29, 49, 21, 57, 62, 73, 54

Solve:

A **max heap** is a type of binary tree where the parent node is always greater than or equal to its child nodes.

23, 29, 49, 21, 57, 62, 73, 54

Represent the numbers as a binary tree (level-wise, left to right):



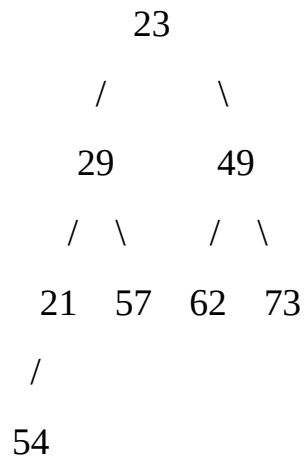
Total Nodes = 08.

Last Non-leaf node index = $(8/2) = 4$.

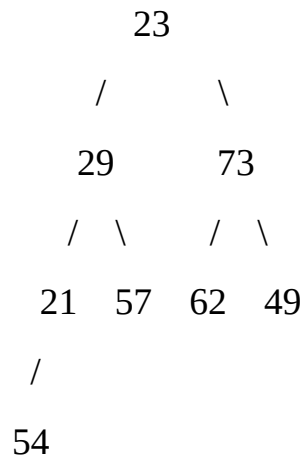
Therefore, last non-leaf node = 21.

To build the heap, heapify only the nodes: [23, 29, 49, 21] in reverse order.

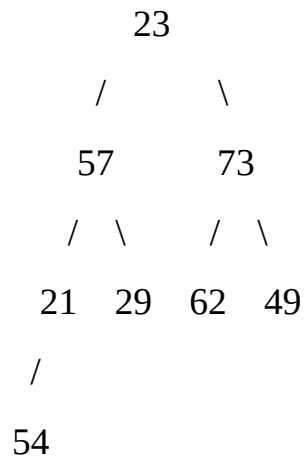
Heapify 21: Swap 21 and 54.



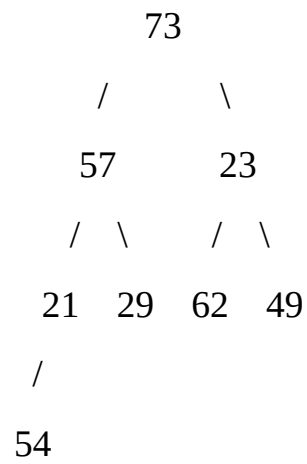
Heapify 49: Swap 49 and 73.



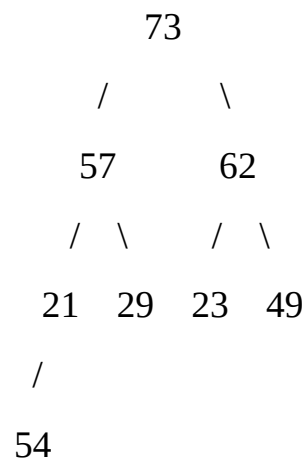
Heapify 29: Swap 29 and 57.



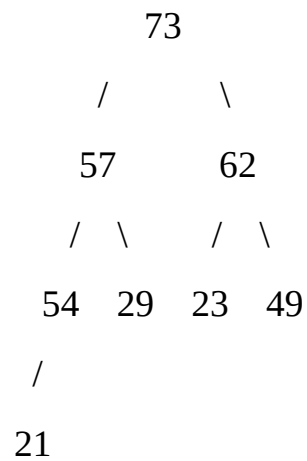
Heapify 23: Swap 23 and 73.



Heapify 23: Swap 23 and 62.



Heapify 21: Swap 21 and 54.



Final Max Heap (Array Representation): [73, 57, 62, 54, 29, 23, 49, 21]

C. Explain which data structure is most efficient to find the top 10 largest items out of 1 million items stored in file? Why?

i) Min heap ii) Max heap iii) BST iv) Sorted array

Solve:

The most efficient data structure is **Min Heap**, as it allows maintaining the top 10 largest items efficiently by keeping the smallest of the top 10 at the root, enabling easy replacement when larger items are encountered.

Spring 2024

4.

A. Analyze the differences between complete binary tree and heap tree with figure. Build a max heap from the following list of numbers:

33, 29, 49, 21, 57, 62, 73, 54

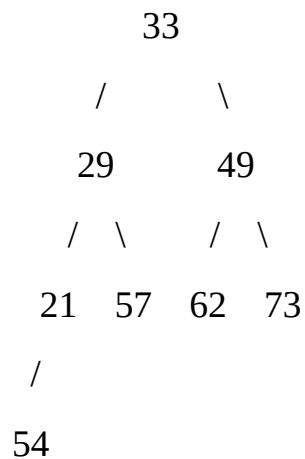
Solve:

Differences Between Complete Binary Tree and Heap Tree

Complete Binary Tree	Heap Tree
All levels filled except possibly the last, filled left to right.	A complete binary tree that satisfies heap property (max or min).
Efficient for traversal and full tree representation.	Used in priority queues and heap sort algorithms.
No specific value requirement, just structure.	Nodes follow heap property.
<pre> 33 / \ 29 49 / \ / \ 21 57 62 73 / 54</pre>	<pre> 73 / \ 57 62 / \ / \ 33 29 49 21 / 54</pre>

33, 29, 49, 21, 57, 62, 73, 54

Represent the numbers as a binary tree (level-wise, left to right):



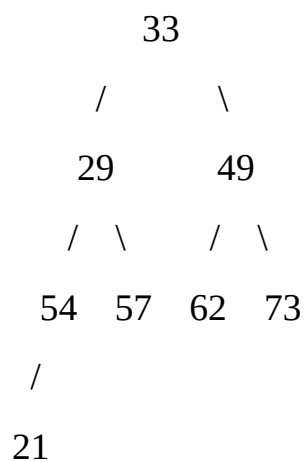
Total Nodes = 08.

Last Non-leaf node index = $(8/2) = 4$.

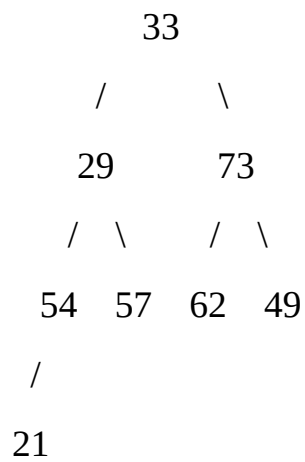
Therefore, last non-leaf node = 21.

To build the heap, heapify only the nodes: [33, 29, 49, 21] in reverse order.

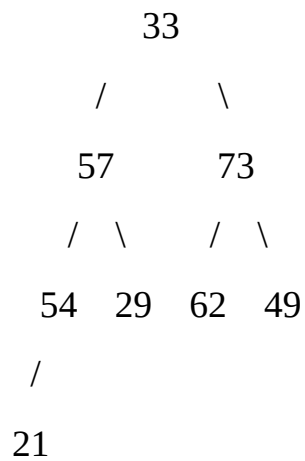
Heapify 21: Swap 21 and 54.



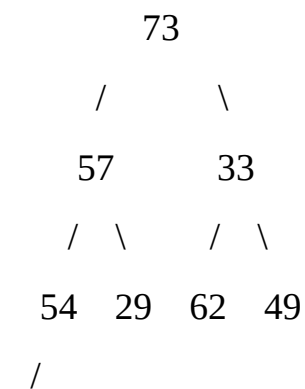
Heapify 49: Swap 49 and 73.



Heapify 29: Swap 29 and 57.

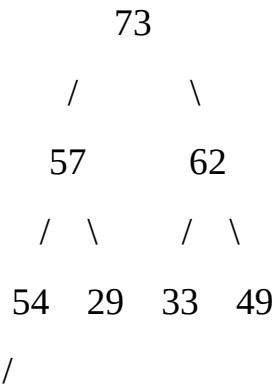


Heapify 33: Swap 33 and 73.



21

Heapify 33: Swap 33 and 62.



21

Final Max Heap (Array Representation): [73, 57, 62, 54, 29, 33, 49, 21]

B. Write the difference between binary search tree and m-way search tree. How do you delete a node from the binary search tree? Explain the deletion with example.

Solve:

Difference Between Binary Search Tree (BST) and M-Way Search Tree:

Aspect	Binary Search Tree (BST)	M-Way Search Tree
Number of Children	Each node has at most 2 children.	Each node can have up to MM children.
Keys per Node	Each node contains exactly 1 key.	Each node can contain up to M-1M-1 keys.
Height	Generally deeper due to binary split.	Typically shallower for the same number of elements.
Traversal Complexity	Traversal is linear (O(n)O(n)).	Traversal depends on MM, faster for large MM.
Use Case	Suitable for small, dynamic data.	Suitable for large datasets like databases.

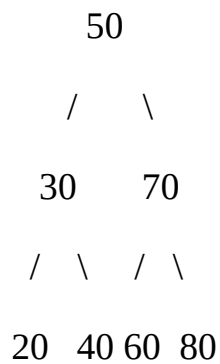
Deleting a Node from a Binary Search Tree (BST)

1. Node to be deleted is a Leaf Node (No Children):

- Simply remove the node from the tree.

Example:

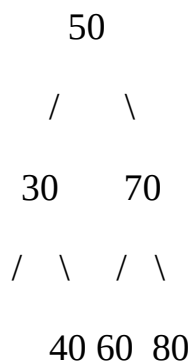
Tree Before Deletion:



Delete 20:

Since 20 is a leaf node, remove it directly.

Tree After Deletion:

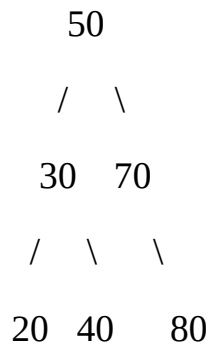


2. Node to be deleted has One Child:

- Replace the node with its child.

Example:

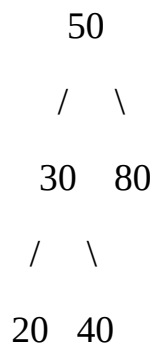
Tree Before Deletion:



Delete 70:

70 has only one child (80). Replace 70 with 80.

Tree After Deletion:

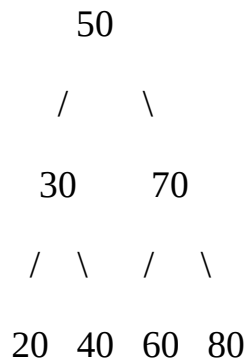


3. Node to be deleted has Two Children:

- Replace the node with either:
 - **Inorder Successor:** The smallest value in the right subtree.
 - **Inorder Predecessor:** The largest value in the left subtree.
- Delete the successor or predecessor node afterward.

Example:

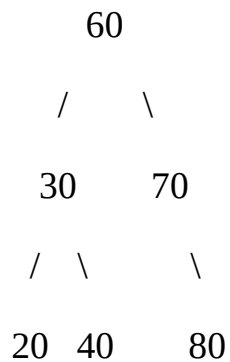
Tree Before Deletion:



Delete 50:

- 50 has two children.
- Find the **inorder successor** (smallest value in the right subtree), which is 60.
- Replace 50 with 60 and then delete 60.

Tree After Deletion:



Illustrate the operation of Heap sort on the array $A=(5,13,2,25,N,7,17,20,8,4)$.
 (Note: N is the sum of last two digits of your ID. i.e. if your ID is C151216, then $N=1+6=7$)

Solve:

$A=(5,13,2,25,12,7,17,20,8,4)$ last digit sum = $9+3 = 12$

Build a Max Heap

5	13	2	25	12	7	17	20	8	4
---	----	---	----	----	---	----	----	---	---

```

      5
     /
    13
    
```

Swap 5 and 13

13	5	2	25	12	7	17	20	8	4
----	---	---	----	----	---	----	----	---	---

```

      25
     /  \
    13   2
     /
    5
    
```

Swap 5 and 25

Swap 25 and 5

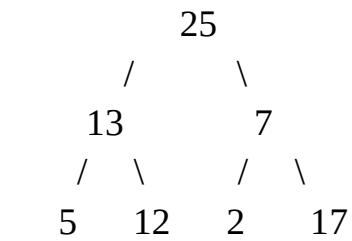
25	13	2	5	12	7	17	20	8	4
----	----	---	---	----	---	----	----	---	---

```

      25
     /  \
    13   2
   /  \  /
  5  12 7
    
```

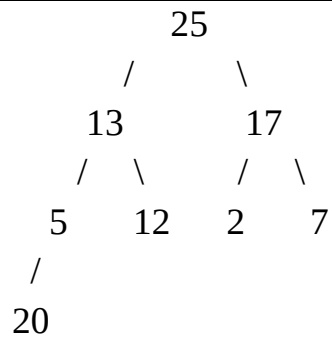
Swap 2 and 7

25	13	7	5	12	2	17	20	8	4
----	----	---	---	----	---	----	----	---	---



Swap 7 and 17

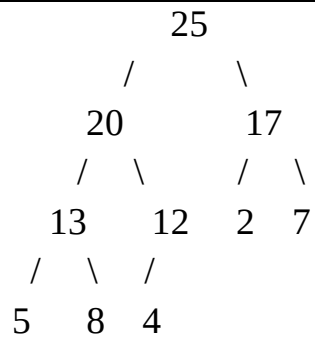
25	13	17	5	12	2	7	20	8	4
----	----	----	---	----	---	---	----	---	---



Swap 5 and 20

Swap 13 and 20

25	20	17	13	12	2	7	5	8	4
----	----	----	----	----	---	---	---	---	---



Array Representation of Max Heap:

25	20	17	13	12	2	7	5	8	4
----	----	----	----	----	---	---	---	---	---

Step 2: Perform Heap Sort

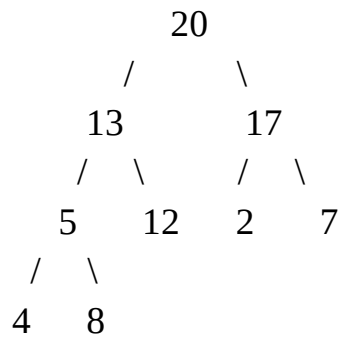
Sorting Process:

1. Remove 25 (root):

- Swap 25 with 4.

- Heapify the reduced heap of size 9.

- New Heap: [20, 13, 17, 5, 12, 7, 2, 4, 8, 25]

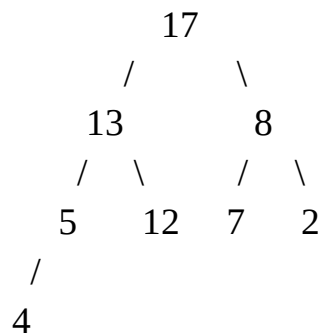


2. Remove 20:

- Swap 20 with 8.

- Heapify the reduced heap of size 8.

- New Heap: [17, 13, 8, 5, 12, 7, 2, 4, 20, 25]

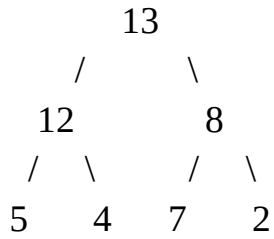


3. Remove 17:

- Swap 17 with 4.

- Heapify the reduced heap of size 7.

- New Heap: [13, 12, 8, 5, 4, 7, 2, 17, 20, 25]



4. Repeat this process until the heap is empty.

Final Sorted Array:

[2, 4, 5, 7, 8, 12, 13, 17, 20, 25]